Hangry Frogs

Generated by Doxygen 1.9.1

1 Hierarchical Index 1
1.1 Class Hierarchy . . . 1

2 Class Index 3
2.1 Class List . . . 3

3 Class Documentation 5
3.1 Button Class Reference . . . 5
3.1.1 Detailed Description . . . 5
3.1.2 Member Function Documentation . . . 5
3.1.2.1 draw_button() . . . 5
3.1.2.2 get_name() . . . 6
3.1.2.3 get_sprite() . . . 6
3.1.2.4 set_position() . . . 6
3.2 Enemy Class Reference . . . 7
3.2.1 Detailed Description . . . 8
3.2.2 Member Function Documentation . . . 8
3.2.2.1 get_checkpoint() . . . 8
3.2.2.2 get_distance_left() . . . 8
3.2.2.3 get_health() . . . 9
3.2.2.4 point_to_path() . . . 9
3.2.2.5 set_slow() . . . 9
3.2.2.6 update() . . . 9
3.3 Entity Class Reference . . . 10
3.3.1 Detailed Description . . . 11
3.3.2 Member Function Documentation . . . 11
3.3.2.1 collision() . . . 11
3.3.2.2 distance() . . . 11
3.3.2.3 draw() . . . 12
3.3.2.4 get_coordinates() . . . 12
3.3.2.5 get_radius() . . . 12
3.3.2.6 update() . . . 12
3.4 Fly Class Reference . . . 13
3.4.1 Detailed Description . . . 14
3.4.2 Member Function Documentation . . . 14
3.4.2.1 update() . . . 14
3.5 Frost_Pepe Class Reference . . . 15
3.5.1 Detailed Description . . . 16
3.5.2 Member Function Documentation . . . 16
3.5.2.1 update() . . . 16
3.6 Game_Engine Class Reference . . . 16
3.6.1 Detailed Description . . . 17
3.7 McFly Class Reference . . . 17

3.7.1 Detailed Description . . . 18
3.7.2 Member Function Documentation . . . 18
3.7.2.1 update() . . . 18
3.8 Pepe Class Reference . . . 18
3.8.1 Detailed Description . . . 19
3.8.2 Member Function Documentation . . . 20
3.8.2.1 update() . . . 20
3.9 Projectile Class Reference . . . 20
3.9.1 Detailed Description . . . 21
3.9.2 Member Function Documentation . . . 21
3.9.2.1 get_projectile_hit() . . . 21
3.9.2.2 update() . . . 22
3.10 Turret Class Reference . . . 22
3.10.1 Detailed Description . . . 23
3.10.2 Member Function Documentation . . . 23
3.10.2.1 attack() . . . 23
3.10.2.2 collision_turrets() . . . 24
3.10.2.3 set_position() . . . 24
3.10.2.4 update() . . . 24
3.11 User_Interface Class Reference . . . 25
3.11.1 Detailed Description . . . 25
3.11.2 Member Function Documentation . . . 25
3.11.2.1 add_button() . . . 25
3.11.2.2 draw_bar() . . . 26
3.11.2.3 on_bar() . . . 26
3.12 World Class Reference . . . 26
3.12.1 Detailed Description . . . 27
3.12.2 Member Function Documentation . . . 27
3.12.2.1 collision() . . . 27
3.12.2.2 draw_background() . . . 28
3.12.2.3 draw_bar() . . . 28
3.12.2.4 draw_entities() . . . 28
3.12.2.5 get_checkpoint() . . . 28
3.12.2.6 get_user_interface() . . . 29
3.12.2.7 place_turret() . . . 29
3.12.2.8 update_entities() . . . 29

Index **31**

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

- Button 5
- Entity 10
  - Enemy 7
    - Fly 13
    - McFly 17
  - Projectile 20
  - Turret 22
    - Frost_Pepe 15
    - Pepe 18
- Game_Engine 16
- User_Interface 25
- World 26

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Button
: Clickable object that returns a string when clicked 5

Enemy
: Abstract class representing enemies that inherits from Entity 7

Entity
: A basic game object, with a position, radius, direction and sprite 10

Fly
: Implementation of the abstract Enemy class 13

Frost_Pepe
: Object that represents a Turret that slows entities of Enemy class 15

Game_Engine
: Responsible for the game loop, updating and drawing the game 16

McFly
: Implementation of the abstract Enemy class, can spawn Fly objects 17

Pepe
: Object that represents a Turret that shoot projectiles 18

Projectile
: Projectile that travels at a constant speed and damages enemies. Inherits Entity 20

Turret
: Abstract class that represents turrets in game. Inherits from Entity 22

User_Interface
: Object to represent the user interface in the game. Contains all button used by the player . . . 25

World
: A class to represent the game 26

Chapter 3

Class Documentation

3.1 Button Class Reference

Clickable object that returns a string when clicked.

```
#include <button.h>
```

Public Member Functions

- Button (sf::Texture const &texture, std::string const &name)

  String to return when clicked.
- void draw_button (sf::RenderWindow &window)

  Draws the button to the screen.
- void set_position (sf::Vector2f const &coordinates)

  Set the position of the button.
- sf::Sprite get_sprite () const

  Get the sprite of the button.
- std::string get_name () const

  Get the name of the button.

3.1.1 Detailed Description

Clickable object that returns a string when clicked.

3.1.2 Member Function Documentation

3.1.2.1 draw_button()

```
void Button::draw_button (
            sf::RenderWindow & window )
```

Draws the button to the screen.

Parameters

window	Window to draw the button on.

3.1.2.2 get_name()

```
std::string Button::get_name ( ) const
```

Get the name of the button.

Returns

std::string

3.1.2.3 get_sprite()

```
sf::Sprite Button::get_sprite ( ) const
```

Get the sprite of the button.

Returns

sf::Sprite

3.1.2.4 set_position()

```
void Button::set_position (
            sf::Vector2f const & coordinates )
```

Set the position of the button.

Parameters

coordinates	Position to set the button to.

The documentation for this class was generated from the following files:

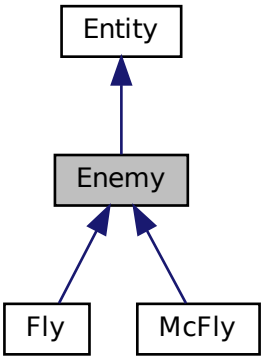
- src/button.h
- src/button.cc

3.2 Enemy Class Reference

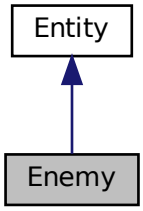
Abstract class representing enemies that inherits from Entity.

```
#include <enemy.h>
```

Inheritance diagram for Enemy:



Collaboration diagram for Enemy:



Public Member Functions

- Enemy (sf::Texture const &texture, sf::Vector2f const &coordinates, float const radius, int const health, int const checkpoint, std::vector< sf::Vector2f > const &path, int const speed)

  Tracks current targeted path checkpoints index.
- virtual void update (sf::Time const &delta) override=0

  Pure virtual, updates the enemy.
- void take_damage ()

  Lowers enemy health by one.
- int get_health () const

Get the health of the enemy.

- float get_distance_left () const

  Get the distance left to the last checkpoint.

- void set_slow (bool const slow)

  Set the slow bool.

- void point_to_path (sf::Time const &delta)

  Points direction vector towards the target checkpoint, sets the magnitude according to speed.

- int get_checkpoint () const

  Get the current checkpoints index.

Protected Attributes

- std::vector< sf::Vector2f > const & path

  Path consiting of a number of checkpoints to pass through.

3.2.1 Detailed Description

Abstract class representing enemies that inherits from Entity.

3.2.2 Member Function Documentation

3.2.2.1 get_checkpoint()

```
int Enemy::get_checkpoint ( ) const
```

Get the current checkpoints index.

Returns

int

3.2.2.2 get_distance_left()

```
float Enemy::get_distance_left ( ) const
```

Get the distance left to the last checkpoint.

Returns

float

3.2.2.3 get_health()

```
int Enemy::get_health ( ) const
```

Get the health of the enemy.

Returns

int

3.2.2.4 point_to_path()

```
void Enemy::point_to_path (
            sf::Time const & delta )
```

Points direction vector towards the target checkpoint, sets the magnitude according to speed.

Parameters

delta	Delta time to relate speed to.

3.2.2.5 set_slow()

```
void Enemy::set_slow (
            bool const slow )
```

Set the slow bool.

Parameters

slow	Bool argument to set slow to.

3.2.2.6 update()

```
virtual void Enemy::update (
            sf::Time const & delta )  [override], [pure virtual]
```

Pure virtual, updates the enemy.

Parameters

delta	Delta time argument.

Implements Entity.

Implemented in McFly, and Fly.

The documentation for this class was generated from the following files:

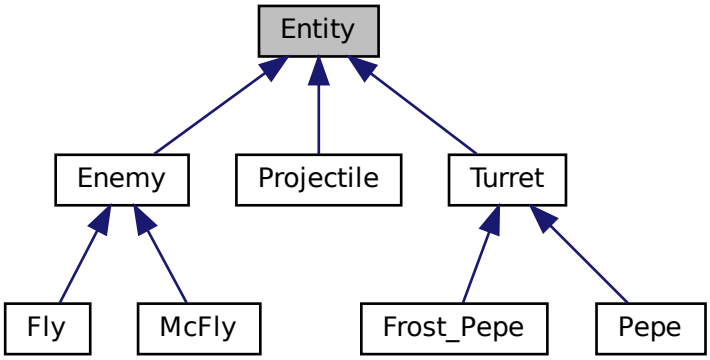
- src/enemy.h
- src/enemy.cc

3.3 Entity Class Reference

A basic game object, with a position, radius, direction and sprite.

```
#include <entity.h>
```

Inheritance diagram for Entity:



Public Member Functions

- Entity (sf::Texture const &texture, sf::Vector2f const &coordinates, float const radius)

  Sprite drawn at the entity position.
- virtual void update (sf::Time const &delta)=0

  Pure virtual, updates entity.
- void draw (sf::RenderWindow &window)

  Draw entity sprite.
- virtual bool collision (float const rad, Entity const &other) const

  Check collision with other entity given a radius.
- float distance (sf::Vector2f const &other) const

  Calculate distance to position.
- sf::Vector2f get_coordinates () const

  Get the coordinate of the entity.
- float get_radius () const

  Get the radius of the entity.

Protected Attributes

- sf::Vector2f **direction**
- sf::Vector2f coordinates

  Direction the entity points in.

3.3.1 Detailed Description

A basic game object, with a position, radius, direction and sprite.

3.3.2 Member Function Documentation

3.3.2.1 collision()

```
bool Entity::collision (
            float const rad,
            Entity const & other ) const  [virtual]
```

Check collision with other entity given a radius.

Parameters

rad	Radius to use.
other	Other entity to collide with.

Returns

true

false

3.3.2.2 distance()

```
float Entity::distance (
            sf::Vector2f const & other ) const
```

Calculate distance to position.

Parameters

other	Other position.

Returns

float

3.3.2.3 draw()

```
void Entity::draw (
            sf::RenderWindow & window )
```

Draw entity sprite.

Parameters

window	Window that is drawn to.

3.3.2.4 get_coordinates()

```
sf::Vector2f Entity::get_coordinates ( ) const
```

Get the coordinate of the entity.

Returns

sf::Vector2f

3.3.2.5 get_radius()

```
float Entity::get_radius ( ) const
```

Get the radius of the entity.

Returns

float

3.3.2.6 update()

```
virtual void Entity::update (
            sf::Time const & delta )  [pure virtual]
```

Pure virtual, updates entity.

Parameters

delta	Delta time argument.

Implemented in Turret, Enemy, Projectile, Pepe, McFly, Frost_Pepe, and Fly.

The documentation for this class was generated from the following files:

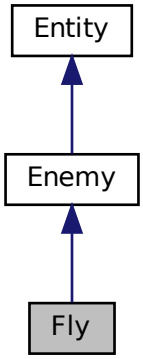
- src/entity.h
- src/entity.cc

3.4 Fly Class Reference

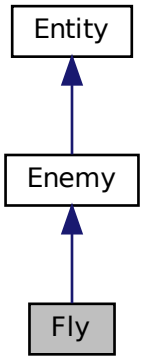
Implementation of the abstract Enemy class.

```
#include <fly.h>
```

Inheritance diagram for Fly:



Collaboration diagram for Fly:



Public Member Functions

- **Fly** (sf::Texture const &texture, sf::Vector2f const &coordinates, int const checkpoint, std::vector< sf::Vector2f > const &path)
- void update (sf::Time const &delta) override

  Moves the object along a path.

Additional Inherited Members

3.4.1 Detailed Description

Implementation of the abstract Enemy class.

3.4.2 Member Function Documentation

3.4.2.1 update()

```
void Fly::update (
            sf::Time const & delta )  [override], [virtual]
```

Moves the object along a path.

Parameters

delta	Delta time to move the object in relation to.

Implements Enemy.

The documentation for this class was generated from the following files:

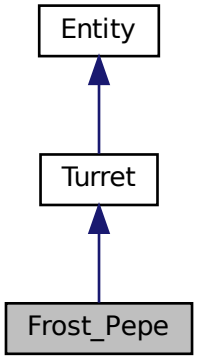
- src/fly.h
- src/fly.cc

3.5 Frost_Pepe Class Reference

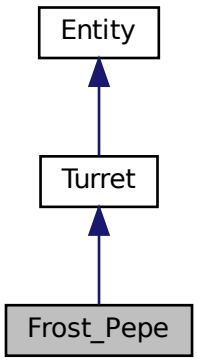
Object that represents a Turret that slows entities of Enemy class.

```
#include <frost_pepe.h>
```

Inheritance diagram for Frost_Pepe:



Collaboration diagram for Frost_Pepe:



Public Member Functions

- **Frost_Pepe** (sf::Texture const &texture, sf::Vector2f const &position, std::vector< Entity * > &entities)
- void update (sf::Time const &delta) override

  Function to update object.

Additional Inherited Members

3.5.1 Detailed Description

Object that represents a Turret that slows entities of Enemy class.

3.5.2 Member Function Documentation

3.5.2.1 update()

```
void Frost_Pepe::update (
            sf::Time const & delta )  [override], [virtual]
```

Function to update object.

Parameters

delta	an sf::Vector2f argument.

Implements Turret.

The documentation for this class was generated from the following files:

- src/frost_pepe.h
- src/frost_pepe.cc

3.6 Game_Engine Class Reference

Responsible for the game loop, updating and drawing the game.

```
#include <game_engine.h>
```

Public Member Functions

- void run ()

  Runs the game loop.

3.6.1 Detailed Description

Responsible for the game loop, updating and drawing the game.

The documentation for this class was generated from the following files:

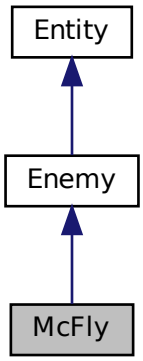
- src/game_engine.h
- src/game_engine.cc

3.7 McFly Class Reference

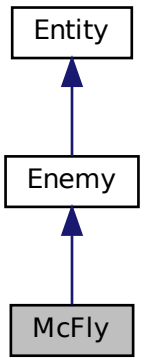
Implementation of the abstract Enemy class, can spawn Fly objects.

```
#include <mcfly.h>
```

Inheritance diagram for McFly:



Collaboration diagram for McFly:



Public Member Functions

- McFly (sf::Texture const &texture, sf::Vector2f coordinates, std::vector< sf::Vector2f > const &path, std::vector< Entity * > &entities, sf::Texture &spawned_texture)

  Texture of the spawned enemy.
- void update (sf::Time const &delta) override

  Moves the object along a path, spawns enemies at a set interval.

Additional Inherited Members

3.7.1 Detailed Description

Implementation of the abstract Enemy class, can spawn Fly objects.

3.7.2 Member Function Documentation

3.7.2.1 update()

```
void McFly::update (
            sf::Time const & delta )  [override], [virtual]
```

Moves the object along a path, spawns enemies at a set interval.

Parameters

delta	Delta time to move the object and spawn enemies in relation to.

Implements Enemy.

The documentation for this class was generated from the following files:

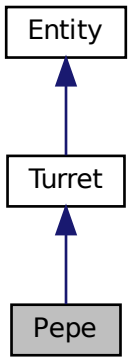
- src/mcfly.h
- src/mcfly.cc

3.8 Pepe Class Reference

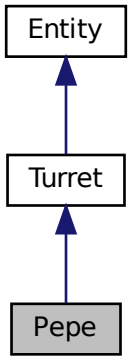
Object that represents a Turret that shoot projectiles.

```
#include <pepe.h>
```

Inheritance diagram for Pepe:



Collaboration diagram for Pepe:



Public Member Functions

- Pepe (sf::Texture const &texture, sf::Vector2f const &position, std::vector< Entity * > &entities, sf::Texture const &projectile_texture)

  Texture of spawned projectile.
- void update (sf::Time const &delta) override

  Function to update object.

Additional Inherited Members

3.8.1 Detailed Description

Object that represents a Turret that shoot projectiles.

3.8.2 Member Function Documentation

3.8.2.1 update()

```
void Pepe::update (
            sf::Time const & delta )  [override], [virtual]
```

Function to update object.

Parameters

delta	an sf::Vector2f argument.

Implements Turret.

The documentation for this class was generated from the following files:

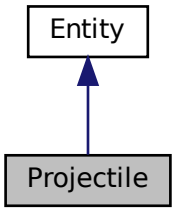
- src/pepe.h
- src/pepe.cc

3.9 Projectile Class Reference

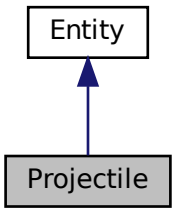
Projectile that travels at a constant speed and damages enemies. Inherits Entity.

```
#include <projectile.h>
```

Inheritance diagram for Projectile:



Collaboration diagram for Projectile:



Public Member Functions

- Projectile (sf::Texture const &texture, sf::Vector2f const &coordinates, float const radius, sf::Vector2f const &velocity, std::vector< Entity * > &entities)

  True if the enemy has hit a target.
- void update (sf::Time const &delta) override

  Moves the projectile at a constant speed, damages enemies hit.
- void hit ()

  Checks if the projectile has hit an enemy. Damages hit enemies and updates projectile_hit.
- bool get_projectile_hit () const

  Get the value of projectile_hit.

Additional Inherited Members

3.9.1 Detailed Description

Projectile that travels at a constant speed and damages enemies. Inherits Entity.

3.9.2 Member Function Documentation

3.9.2.1 get_projectile_hit()

```
bool Projectile::get_projectile_hit ( ) const
```

Get the value of projectile_hit.

Returns

true

false

3.9.2.2 update()

```
void Projectile::update (
            sf::Time const & delta )  [override], [virtual]
```

Moves the projectile at a constant speed, damages enemies hit.

Parameters

delta	Delta time to move the projectile in relation to.

Implements Entity.

The documentation for this class was generated from the following files:

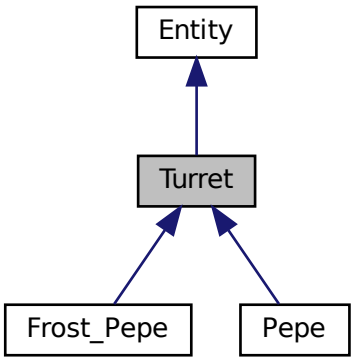
- src/projectile.h
- src/projectile.cc

3.10 Turret Class Reference

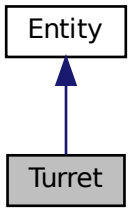
Abstract class that represents turrets in game. Inherits from Entity.

```
#include <turret.h>
```

Inheritance diagram for Turret:



Collaboration diagram for Turret:



Public Member Functions

- **Turret** (sf::Texture const &texture, sf::Vector2f const &coordinates, float const radius, float const attack_radius, std::vector< Entity * > &entities)
- virtual void update (sf::Time const &delta) override=0

  Function to update object.
- virtual bool collision_turrets (int const rad) const

  Function to deter if user is trying place a turret on another turret in game.
- void set_position (sf::Vector2f const &coord)

  Function to set position of turret.

Protected Member Functions

- virtual void attack (sf::Time const &delta)=0

  The turrets attack radius.

Protected Attributes

- std::vector< Entity * > & **entities**
- float attack_radius

  Vector of entities used for attack and collision.

3.10.1 Detailed Description

Abstract class that represents turrets in game. Inherits from Entity.

3.10.2 Member Function Documentation

3.10.2.1 attack()

```
virtual void Turret::attack (
            sf::Time const & delta )  [protected], [pure virtual]
```

The turrets attack radius.

Function to attack entities of Enemy class if there's a collision.

Parameters

delta	an sf::Time argument.

3.10.2.2 collision_turrets()

```
bool Turret::collision_turrets (
            int const rad ) const  [virtual]
```

Function to deter if user is trying place a turret on another turret in game.

Parameters

rad	an int argument based on object radius.

Returns

true

false

3.10.2.3 set_position()

```
void Turret::set_position (
            sf::Vector2f const & coord )
```

Function to set position of turret.

Parameters

coord	an sf::Vector2f argument.

3.10.2.4 update()

```
virtual void Turret::update (
            sf::Time const & delta )  [override], [pure virtual]
```

Function to update object.

Parameters

delta	an sf::Vector2f argument.

Implements Entity.

Implemented in Pepe, and Frost_Pepe.

The documentation for this class was generated from the following files:

- src/turret.h
- src/turret.cc

3.11 User_Interface Class Reference

Object to represent the user interface in the game. Contains all button used by the player.

```
#include <user_interface.h>
```

Public Member Functions

- **User_Interface** (sf::Texture const &texture, sf::Vector2f const &coordinates)
- void draw_bar (sf::RenderWindow &window)

  Function to display the user interface bar in game.
- void add_button (sf::Texture const &texture, std::string const &name)

  Function to store a Button object in a specific data container.
- bool on_bar (sf::RenderWindow &window, std::string &turret_name)

  Function to deter if the user have clicked on the user interface bar.

3.11.1 Detailed Description

Object to represent the user interface in the game. Contains all button used by the player.

3.11.2 Member Function Documentation

3.11.2.1 add_button()

```
void User_Interface::add_button (
            sf::Texture const & texture,
            std::string const & name )
```

Function to store a Button object in a specific data container.

Parameters

texture	an sf::Texture argument containting the buttons texture.
name	a std::string argument. The name of the button.

3.11.2.2 draw_bar()

```
void User_Interface::draw_bar (
            sf::RenderWindow & window )
```

Function to display the user interface bar in game.

Parameters

window	an sf::RenderWindow argument.

3.11.2.3 on_bar()

```
bool User_Interface::on_bar (
            sf::RenderWindow & window,
            std::string & turret_name )
```

Function to deter if the user have clicked on the user interface bar.

Parameters

window	an sf::RenderWindow. Used to get coordinates from mouse click.
turret_name	an std::string by reference. Name changes based on the selected button.

Returns

true

false

The documentation for this class was generated from the following files:

- src/user_interface.h
- src/user_interface.cc

3.12 World Class Reference

A class to represent the game.

```
#include <world.h>
```

Public Member Functions

- World ()

  Return value of the last button that was pressed.
- void draw_background (sf::RenderWindow &window)

  Renders the background image.
- void draw_entities (sf::RenderWindow &window)

  Function to display all the entitites stored in a specific data container.
- void draw_bar (sf::RenderWindow &window)

  Function to display the actionbar in the game that the user interacts with to build turrets.
- void update_entities (sf::Time const &delta)

  Function to update all entities inside the class.
- void place_turret (sf::RenderWindow &window)

  Function to store a new Turret class object into a specific data container.
- std::string & **get_button_reference** ()
- sf::Vector2f get_checkpoint (int constindex) const

  Function that returns a certain coordinate pair on index in a specific data container wich represent the path that is the displayed in the game.
- User_Interface & get_user_interface ()

  Function that return the actionbar objects by reference.
- bool collision (sf::Vector2f const &entity_coord, int const entity_rad)

  Collision between an entity and the path.

3.12.1 Detailed Description

A class to represent the game.

3.12.2 Member Function Documentation

3.12.2.1 collision()

```
bool World::collision (
            sf::Vector2f const & entity_coord,
            int const entity_rad )
```

Collision between an entity and the path.

Parameters

entity_coord	Entity position.
entity_rad	Entity radius.

Returns

true

false

3.12.2.2 draw_background()

```
void World::draw_background (
            sf::RenderWindow & window )
```

Renders the background image.

Parameters

window	an sf::RenderWindow argument.

3.12.2.3 draw_bar()

```
void World::draw_bar (
            sf::RenderWindow & window )
```

Function to display the actionbar in the game that the user interacts with to build turrets.

Parameters

window	an sf::RenderWindow argument.

3.12.2.4 draw_entities()

```
void World::draw_entities (
            sf::RenderWindow & window )
```

Function to display all the entitites stored in a specific data container.

Parameters

window	an sf::RenderWindow argument.

3.12.2.5 get_checkpoint()

```
sf::Vector2f World::get_checkpoint (
            int constindex ) const
```

Function that returns a certain coordinate pair on index in a specific data container wich represent the path that is the displayed in the game.

Parameters

index	an integer argument.

Returns

sf::Vector2f The coordinate

3.12.2.6 get_user_interface()

```
User_Interface & World::get_user_interface ( )
```

Function that return the actionbar objects by reference.

Returns

Actionbar& The Action bar class.

3.12.2.7 place_turret()

```
void World::place_turret (
            sf::RenderWindow & window )
```

Function to store a new Turret class object into a specific data container.

Parameters

window	an sf::Vector2f argument.

3.12.2.8 update_entities()

```
void World::update_entities (
            sf::Time const & delta )
```

Function to update all entities inside the class.

Parameters

delta	- an sf::Time argument.

The documentation for this class was generated from the following files:

- src/world.h
- src/world.cc

Index

add_button
User_Interface, 25
attack
Turret, 23

Button, 5
draw_button, 5
get_name, 6
get_sprite, 6
set_position, 6

collision
Entity, 11
World, 27
collision_turrets
Turret, 24

distance
Entity, 11
draw
Entity, 12
draw_background
World, 28
draw_bar
User_Interface, 26
World, 28
draw_button
Button, 5
draw_entities
World, 28

Enemy, 7
get_checkpoint, 8
get_distance_left, 8
get_health, 8
point_to_path, 9
set_slow, 9
update, 9
Entity, 10
collision, 11
distance, 11
draw, 12
get_coordinates, 12
get_radius, 12
update, 12

Fly, 13
update, 14
Frost_Pepe, 15
update, 16

Game_Engine, 16
get_checkpoint
Enemy, 8
World, 28
get_coordinates
Entity, 12
get_distance_left
Enemy, 8
get_health
Enemy, 8
get_name
Button, 6
get_projectile_hit
Projectile, 21
get_radius
Entity, 12
get_sprite
Button, 6
get_user_interface
World, 29

McFly, 17
update, 18

on_bar
User_Interface, 26

Pepe, 18
update, 20
place_turret
World, 29
point_to_path
Enemy, 9
Projectile, 20
get_projectile_hit, 21
update, 21

set_position
Button, 6
Turret, 24
set_slow
Enemy, 9

Turret, 22
attack, 23
collision_turrets, 24
set_position, 24
update, 24

update
Enemy, 9

Entity, 12
Fly, 14
Frost_Pepe, 16
McFly, 18
Pepe, 20
Projectile, 21
Turret, 24
update_entities
World, 29
User_Interface, 25
add_button, 25
draw_bar, 26
on_bar, 26

World, 26
collision, 27
draw_background, 28
draw_bar, 28
draw_entities, 28
get_checkpoint, 28
get_user_interface, 29
place_turret, 29
update_entities, 29